

Explicando Flutter pro meu Amor

by Marietti

Objetivos

Com a escrita desse documento, o código ficou mais claro, explicado e organizado, detalhes foram percebidos, e, assim, ficará mais fácil da outra mexer e entender o código. Além disso, servirá como base para a prova. Recomendo que, após a leitura desse documento, dê uma olhada no código real, pois aqui os comentários ficam quebrados, e alguns trechos foram recortados, pois foram considerados repetidos e só aumentaria o tempo de leitura desse documento, dificultando seu entendimento.

PS.: Fonte Arial.

Sumário

Objetivos.....	1
Sumário.....	1
Configuração e instalação do Flutter.....	2
Imports.....	2
Inicialização.....	2
Como começar uma classe?.....	3
Child e Children.....	3
Layouts.....	4
Elementos de UI Principais.....	4
Decorações Comuns.....	4
Imagem.....	5
Navegação de Telas.....	5
Detalhes, Padrões e indentação.....	5
Na Prática: Telas Reutilizáveis.....	5
BaseBloqueio.....	5
BaseInicial.....	7
MostraProdutos.....	9
Telas de Fato.....	12
TelaBloqueio.....	12
TelaLogin - TelaCadastro.....	14
TelaInicial.....	18
TelaProduto.....	18
TelaPesquisar.....	23
TelaConta.....	24
TelaConfiguracoes.....	28
TelaWishlist - TelaDegustados.....	32
TelaAvaliar.....	33

Configuração e instalação do Flutter

Precisa configurar o android studio para funcionar o flutter no VSC.

Na página inicial do Android Studio clicar nos 3 pontinhos ->

sdk manager ->

sdk tools ->

5 primeiros, pula um, mais 3 com confirma ->

ok

<https://flutter.dev/> -> get started -> windows -> android -> download and install -> install

Mover download flutter p docs e descompactar.

cmd:

```
C:\Users\u24140>set PATH=%PATH%;C:\Users\u24140\Documents\flutter\bin
```

```
flutter doctor --android-licenses
```

```
C:\Users\u24140\Documents\flutter\bin>flutter doctor
```

```
Users\uXXXXX\Documents\flutter\flutter config --no-analytics
```

No VSC: baixar extensões flutter.

Configurações do VSC -> settings -> flutter sdk -> edit settings.json -> "dart.flutterSdkPath":

"C:/Users/u24140/Documents/flutter/bin"

ctrl + shift + p -> flutter new project -> application

run -> no device -> chrome

O arquivo principal fica em lib/main.dart.

Imports

```
import 'package:flutter/material.dart';  
import 'package:url_launcher/url_launcher.dart';
```

O primeiro pacote já vem quando você cria um projeto, o segundo eu precisei adicionar para que fosse possível abrir um link no google.

Inicialização

“main” é o primeiro método que o projeto executa e também já vem quando você cria um projeto. Ele diz para rodar a classe MyApp, e a classe MyApp diz para começar a execução em TelaBloqueio.

```
void main() {  
  // inicia a aplicação com a classe MyApp.  
  runApp(const MyApp());  
}  
  
class MyApp extends StatelessWidget {  
  //construtor da classe  
  const MyApp({super.key});
```

```
// construi um widget nesse contexto
@override
Widget build(BuildContext context) {
  // MaterialApp é o widget raiz que define o tema e a navegação.
  return MaterialApp(home: TelaBloqueio());
}
```

No Flutter, tudo é um widget.

Nem todas as classes necessitam de construtor.

```
const MyApp({super.key});
```

As chaves {} dizem que eles são opcionais, o super diz que ele é passado diretamente para a sua superclasse StatelessWidget, e key é, por padrão, do tipo null, a não ser que seja required, como outros parâmetros de outras classes que veremos daqui a pouco.

StatelessWidget

Diz que a classe não é mutável, os valores não irão mudar.

StatefulWidget

Classe mutável, utilizamos o código quando precisávamos guardar algum valor para passar para a API, bem comum com TextField, mais explicação abaixo.

Como começar uma classe?

Comece escrevendo a base de uma classe:

```
class NomeClasse extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return Layout;
  }
}
```

Logo, você define qual layout você quer usar nesse return, ou mesmo se quer usar uma classe reutilizável. Assim, você vai adicionando os elementos e layouts necessários.

Child e Children

O nome child (filho, em inglês) é uma convenção muito comum no Flutter para se referir a um widget que será aninhado dentro de outro. child: ou children[] são muito usados no código, alguns widgets pedem child e outros children. Como saber? Eu fui digitando chil e vendo o que o VSC me indicava.

child: -> o widget só poderá ter um único filho, caso queira ter mais, use outro widget que use children.

children[] -> tem vários filhos, ilimitado.

Layouts

- Scaffold: Fornece uma estrutura básica de Material Design para uma tela, incluindo propriedades como backgroundColor.
- Stack: Permite que você organize widgets uns sobre os outros, como uma pilha. Útil para backgrounds ou elementos flutuantes.
- Column: Organiza widgets verticalmente em uma única coluna.
- Row: Organiza widgets horizontalmente em uma única linha.
- Center: Centraliza seu widget filho.
- Expanded: Faz com que um widget filho de Row, Column ou Flex ocupe todo o espaço disponível restante.

Elementos de UI Principais

- Container: Um widget de conveniência que combina capacidades de pintura, posicionamento e dimensionamento.
- GridView: Monta uma tabela.
- Padding: Adiciona preenchimento (espaçamento interno) ao redor de um widget.
- Opacity: Define a opacidade do elemento que está dentro dele.
- SingleChildScrollView: Torna um widget filho rolável quando o conteúdo excede o espaço disponível.
- ClipRRect: Recorta o filho em um retângulo com cantos arredondados.
- Align: Posiciona um widget filho dentro de si, de acordo com um Alignment específico.
- Size: Define o tamanho do elemento de dentro.
- SizedBox: Cria caixas de tamanho fixo, úteis para espaçamento vertical ou horizontal.
- Positioned: Usado em Stacks para controlar a posição de um widget filho.
- ElevatedButton: Um botão com fundo e elevação.
- GestureDetector: Detecta gestos do usuário, como toques (onTap), em seu widget filho.
- Text: Exibe uma string de texto.
- TextField: Um campo de entrada de texto que permite ao usuário digitar.
- TextEditingController: Controla e obtém o texto digitado em um TextField.
- TextButton: Um botão de texto simples, sem fundo.

Decorações Comuns

```
padding: const EdgeInsets.all(20.0),
width: 300.0,
decoration BoxDecoration(
  color: const Color.fromRGBO(255, 255, 255, 0.4),
  //ou
  color: Color.white,
  borderRadius: BorderRadius.circular(20),
  boxShadow: [
    BoxShadow(
      color: Colors.black.withOpacity(0.1),
      blurRadius: 10,
      offset: const Offset(0, 5),
    ),
  ],
  ...
```

)

Imagem

```
Image.asset(  
  "assets/fundo.png",  
  fit: BoxFit.cover,  
  height: double.infinity,  
  width: double.infinity,  
) ,
```

- Image.asset: Exibe uma imagem de um arquivo local (assets/fundo.png).
- Image.network (NetworkImage): Exibe uma imagem de um URL da internet.

Navegação de Telas

```
Navigator.push(  
  context,  
  MaterialPageRoute(builder: (context) => NomeDaTela())  
)
```

-> Este é o padrão para empurrar uma nova rota (tela) para a pilha de navegação, levando o usuário para a NomeDaTela. Quando o usuário retorna (por exemplo, pressionando o botão "Voltar"), a tela anterior é exibida.

Detalhes, Padrões e indentação

Alguns fechamentos de parênteses precisarão de vírgulas, outros de ponto-vírgula, e outros de nada.

Às vezes o elemento não possui o padding, e é preciso colocar dentro de outro elemento. Teve elementos que só usei para usar uma decoração específica.

A indentação aqui é diferente, ela parece ser menor.

O nosso código utiliza alguns padrões, como, por exemplo, depois de uma tela tem duas linhas para a próxima. Separa os elementos com uma linha, comentários indentados, todas as telas são TelaNome,

Na Prática: Telas Reutilizáveis

No Gustus, as telas são classes. Todas as telas têm o mesmo fundo e alguns elementos em comum, logo, eu fiz classes reutilizáveis, isso facilita na hora de trocar algo, pois você já troca tudo automaticamente. Basicamente, eu dou um return dessa classe em outra e onde eu adiciono o child, é onde ficará o conteúdo não-reutilizável da outra classe.

BaseBloqueio

Essa daqui é a base com o fundo e uma caixa flutuante:

```
// widgets reutilizável  
class BaseBloqueio extends StatelessWidget {  
  //final indica que a variável child só pode ser inicializada uma única vez e não pode  
  ser alterado depois  
  //child pode ser qualquer outro widget do Flutter que será aninhado dentro de outro
```

```

final Widget child;

//construtor da classe e precisa necessariamente ter um child
const BaseBloqueio({
  required this.child,
});

@override
Widget build(BuildContext context) {
  // TelaBloqueio retorna um Scaffold, que fornece a estrutura básica.
  return Scaffold(
    backgroundColor: const Color.fromRGBO(30, 43, 64, 1.0),

    //stack é um tipo de layout mais flexível
    body: Stack(
      children: [
        // imagem de fundo com opacidade.
        Opacity(
          opacity: 1,
          child: Image.asset(
            "assets/fundo.png",
            fit: BoxFit.cover,          //vai encaixar na tela
            height: double.infinity,
            width: double.infinity,
          ),
        ),

        // centraliza o conteúdo (o Container) na tela.
        Center(
          child: Container(
            width: 300.0,                //não tem unidade de
medida nada aqui
            padding: const EdgeInsets.all(20.0),
            decoration: BoxDecoration(
              color: const Color.fromRGBO(255, 255, 255, 0.4), // fundo translúcido.
              borderRadius: BorderRadius.circular(20),
              boxShadow: [
                BoxShadow(
                  color: Colors.black.withOpacity(0.1),
                  blurRadius: 10,          //sombra desfocada
                  offset: const Offset(0, 5), //direção da sombra
                ),
              ],
            ),
          ),
        ),
      ],
    ),
  );
}

```

```

        child: child,          //virao aqui o resto da tela (child) como filho
(child) do container
      ),
    )
  ],
),
);
}
}

```

BaseInicial

Essa daqui é a base para a maioria das telas, que contém o header/o menu de navegação:

```

class BaseInicial extends StatelessWidget {
  final Widget child;

  const BaseInicial({
    required this.child,
  });

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: const Color.fromRGBO(30, 43, 64, 1),

      body: Stack(
        children: [
          // background com opacidade.
          Opacity(
            opacity: 1,
            child: Image.asset(
              "assets/fundo.png",
              fit: BoxFit.cover,
              height: double.infinity,
              width: double.infinity,
            ),
          ),

          Padding(
            padding: const EdgeInsets.all(10),
            child: Row(
              mainAxisAlignment: MainAxisAlignment.center, // centraliza
horizontalmente

```

```

children: [

    //home
    ElevatedButton(
        //quando pressionado navega para a tela inicial
        onPressed: () {
            Navigator.push(
                context,
                MaterialPageRoute(builder: (context) => TelaInicial()),
            );
        },
        style: ElevatedButton.styleFrom(
            backgroundColor: Colors.transparent, // botao transparente
            shadowColor: Colors.transparent,    // botao sem sombra
            elevation: 0,                        // não tem elevação -> nao
tem sombra
        ),
        child: ClipRRect(                        //usado para recortar (ou "clipar") seu
widget filho em um retângulo com cantos arredondados
            child: Image.asset(
                "assets/home.png",
                height: MediaQuery.of(context).size.height * 0.05, // a altura é
5% da média do tamanho da altura da tela
                fit: BoxFit.cover,
            ),
        ),
    ),

    //outros icones do menu de navegação que só mudam a imagem ent eu
resumi

    //search
    ElevatedButton()

    //profile
    ElevatedButton()

    //settings
    ElevatedButton()

    // o resto do conteúdo será posicionado completamente para preencher a tela
com uma distância de 80 do topo
    Positioned.fill(
        top: 80,
        child: child,

```



```

    ),
  ],
),
);
}
}

```

MostraProdutos

Agora, a classe reutilizável mais complicada de fazer, porém quase todas as telas utilizam essa classe. MostraProdutos mostra os produtos, mas usa um parâmetro de lista, ou seja, ele pode mostrar produtos diferentes (listas diferentes). A tela Wishlist pode instanciar com os produtos que estão na wishlist e a tela Degustados pode instanciar com os produtos que estão na degustados. Isso dependerá da conexão com a API, por enquanto temos uma lista provisória como exemplo.

```

class MostraProdutos extends StatelessWidget {
  // aqui recebe uma lista de produtos futuramente
  final List<Map<String, String>> produtos; // lista de 'maps' como parametro, ou seja
  cada elemento da lista tem um nome de tabela (string) com um dado (string), como no
  exemplo

  // a lista será dada como vazia se nada for passado
  const MostraProdutos({Key? key, this.produtos = const []}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    // exemplo de lista de produtos
    final produtos = [
      {
        "nome": "Risoto de camarão",
        "imagem":
"https://raw.githubusercontent.com/mariwgh/Gustus/refs/heads/main/imagens/risoto-de-camarao.png",
        "descricao": "Esse prato popular tem suas raízes na história do risoto italiano, que surgiu na região da Lombardia durante o século XI, influenciado pelos sarracenos.",
        "linkReceita":
"https://www.tudogostoso.com.br/receita/185493-risoto-de-camarao-sem-frescura.html"
      },
      {
        "nome": "Ratatouille",
        "imagem":
"https://raw.githubusercontent.com/mariwgh/Gustus/refs/heads/main/imagens/ratatouille.png",
        "descricao": "Esse prato popular tem suas raízes na história do risoto italiano, que surgiu na região da Lombardia durante o século XI, influenciado pelos sarracenos.",

```

```

        "linkReceita":
"https://www.tudogostoso.com.br/receita/185493-risoto-de-camarao-sem-frescura.html"
    },
    {
        "nome": "Sushi",
        "imagem":
"https://raw.githubusercontent.com/mariwgh/Gustus/refs/heads/main/imagens/sushi.png",
        "descricao": "Esse prato popular tem suas raízes na história do risoto italiano,
que surgiu na região da Lombardia durante o século XI, influenciado pelos sarracenos.",
        "linkReceita":
"https://www.tudogostoso.com.br/receita/185493-risoto-de-camarao-sem-frescura.html"
    },
];

return GridView.builder(
    padding: const EdgeInsets.all(50),
    shrinkWrap: true, // importante quando está dentro de outra
coluna, pois dimensiona-se para o tamanho mínimo sem brigas com outros elementos
    gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
        crossAxisCount: 2, // duas colunas
        mainAxisSpacing: 150, //espaco
        crossAxisSpacing: 150, //espaco
        childAspectRatio: 1, // mantém quadrado
    ),
    itemCount: produtos.length, //quantos items terao na tabela
    itemBuilder: (context, index) {
        final produto = produtos[index]; // percorre cada elemento da lista o
transformando em produto

        //cada produto é clicável, e quando clica, vai para a tela de seu produto com
mais informacoes
        return GestureDetector(
            onTap: () {
                // navega para a TelaProduto passando os dados (parametros) que serão
mostrados

                Navigator.push(
                    context,
                    MaterialPageRoute(
                        builder: (context) => TelaProduto(
                            nome: produto["nome"]!,
                            imagem: produto["imagem"]!,
                            descricao: produto["descricao"]!,
                            linkReceita: produto["linkReceita"]!,
                        ),
                    ),
                );
            },
        );
    },
);

```

```

    ),
  );
},
child: LayoutBuilder(
  builder: (context, constraints) {
    final largura = constraints.maxWidth; //informa o tamanho máximo e mínimo
    de largura e altura que o widget pai permite para este LayoutBuilder
    final altura = constraints.maxHeight;

    // a caixa que contem os dados de cada produto
    return Container(
      decoration: BoxDecoration(
        color: const Color.fromRGBO(255, 255, 255, 0.4),
        borderRadius: BorderRadius.circular(20),
        boxShadow: [
          BoxShadow(
            color: Colors.black.withOpacity(0.3),
            blurRadius: 5,
            offset: const Offset(1, 5),
          ),
        ],
      ),
      child: Stack(
        clipBehavior: Clip.none,
        children: [
          //imagem do produto em sua vez da lista
          Positioned(
            top: -altura * 0.15, // 15% saindo pra cima
            right: -largura * 0.15, // 15% saindo pro lado
            child: Image.network( //imagem da internet
              produto["imagem"]!,
              width: largura * 0.4, // 40% do container
              fit: BoxFit.cover,
            ),
          ),
          Align(
            alignment: Alignment.bottomLeft, //alinha para baixo a
            esquerda
            child: Padding(
              padding: EdgeInsets.all(largura * 0.05), // 5% de padding
              child: Text(
                produto["nome"]!,
                style: TextStyle(
                  color: Colors.white,

```

```

        fontSize: largura * 0.08, // fonte proporcional
8%
        fontWeight: FontWeight.bold, //negrito
    ),
  ),
),
),
],
),
);
},
)
);
},
);
}
}

```

Telas de Fato

TelaBloqueio

A TelaBloqueio como exemplo de como usar as classes reutilizáveis:

```

class TelaBloqueio extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    // TelaBloqueio retorna um Scaffold, que fornece a estrutura básica
    return BaseBloqueio(
      // texto e botoes
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min, // ocupa o mínimo de espaço
vertical.
        mainAxisAlignment: MainAxisAlignment.center, //alinha no centro

        children: [
          Padding(
            padding: EdgeInsets.all(5),
            child: Row(
              mainAxisAlignment: MainAxisAlignment.start, //usei row so para poder
colocar o texto a esquerda
              children: [
                const Text(
                  "Gustus",

```

```

        style: TextStyle(
          fontSize: 48,
          fontWeight: FontWeight.bold,
          color: Colors.white,
        ),
      ),
    ],
  ),
),
const SizedBox(height: 25), //espaço vertical entre um
elemento e outro

// botao de entrar
ElevatedButton(
  onPressed: () {
    Navigator.push(
      context,
      MaterialPageRoute(builder: (context) => TelaLogin()),
    );
  },
  style: ElevatedButton.styleFrom(
    backgroundColor: const Color.fromRGBO(188, 192, 198, 1),
    foregroundColor: Colors.black, //cor do texto e icone que
estiverem dentro do botao
    minimumSize: const Size(double.infinity, 50),
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(10),
    ),
    shadowColor: Colors.black.withOpacity(0.1),
    elevation: 5, //elevacao de 5, forma uma
sombra maior
  ),
  child: const Text("Entrar"),
),
const SizedBox(height: 15),

// botao de cadastrar
ElevatedButton(
  onPressed: () {
    Navigator.push(
      context,
      MaterialPageRoute(builder: (context) => TelaCadastro()),
    );
  },
),

```

```

        style: ElevatedButton.styleFrom(
            backgroundColor: const Color.fromRGBO(188, 192, 198, 1),
            foregroundColor: Colors.black,
            minimumSize: const Size(double.infinity, 50),
            shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(10),
            ),
            shadowColor: Colors.black.withOpacity(0.1),
            elevation: 5,
        ),
        child: const Text("Cadastrar"),
    ),
    const SizedBox(height: 20),
],
),
);
}
}

```

TelaLogin - TelaCadastro

A tela de login e a tela de cadastro são basicamente as mesmas, só muda os campos e a função mandada pro SQL, então irei colocar aqui somente a de login, por ser mais curta, e a de cadastro também será comentada, mas somente no código mesmo.

Aqui vemos algo novo: StatefulWidget. Usamos ele quando precisamos pegar e armazenar valores mutáveis, como o usuário e senha.

```

class TelaLogin extends StatefulWidget {                                // estado mutável dos campos
    // que o usuario digita
    @override
    State<TelaLogin> createState() => _TelaLoginState();           //objeto de estado deve ser
    // gerenciado para TelaLogin
}

class _TelaLoginState extends State<TelaLogin> {
    // declarando controladores para pegar o texto de cada campo -> controlam e obtem o
    // texto digitado em campos de entrada de texto
    final TextEditingController _userController = TextEditingController();
    final TextEditingController _passwordController = TextEditingController();

    // limpa os controladores quando a tela é chamada
    @override
    void dispose() {
        _userController.dispose();
        _passwordController.dispose();
    }
}

```

```

    super.dispose();
  }

  // função que será chamada quando o botão "Entrar" for pressionado e definira as
variaveis
  void _verificarUsuario() {
    final String usuario = _userController.text;
    final String senha = _passwordController.text;
    // aqui ele chama outra funcao q manda as variaveis p banco de dados
  }

  @override
  Widget build(BuildContext context) {
    // TelaBloqueio retorna um Scaffold, que fornece a estrutura básica.
    return BaseBloqueio(
      // texto e botoes
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,          // ocupa o mínimo de espaço
vertical.
        mainAxisAlignment: MainAxisAlignment.center,    //alinha no centro

        children: [
          Padding(
            padding: EdgeInsets.all(5),
            child: Row(
              mainAxisAlignment: MainAxisAlignment.start, //usei row so para poder
colocar o texto a esquerda
              children: [
                const Text(
                  "Login",
                  style: TextStyle(
                    fontSize: 40,
                    fontWeight: FontWeight.bold,
                    color: Colors.white,
                  ),
                ),
              ],
            ),
          ),
          const SizedBox(height: 20),

          TextField(
            controller: _userController,                  //define a variavel que
sera usada para ir para o bd

```

```

        decoration: InputDecoration(                                //decoracao de lugar que
digita
        labelText: "User",                                       //texto do campo para
digitar
        labelStyle: TextStyle(color: Colors.white),
        enabledBorder: UnderlineInputBorder(                     //a borda sera em baixo,
como uma linha para escrever
        borderSide: BorderSide(color: Colors.white),
        ),
        focusedBorder: UnderlineInputBorder(                     //e quando o usuario
clicar, essa borda sera assim (igual)
        borderSide: BorderSide(color: Colors.white),
        ),
        style: TextStyle(color: Colors.white),
    ),
    const SizedBox(height: 15),

    TextField(
        controller: _passwordController,
        decoration: InputDecoration(
            labelText: "Password",
            labelStyle: TextStyle(color: Colors.white),
            enabledBorder: UnderlineInputBorder(
                borderSide: BorderSide(color: Colors.white),
            ),
            focusedBorder: UnderlineInputBorder(
                borderSide: BorderSide(color: Colors.white),
            ),
        ),
        style: TextStyle(color: Colors.white),
    ),
    const SizedBox(height: 25),

    // botao de entrar
    ElevatedButton(
        onPressed: () {
            _verificarUsuario();                                //funcao que chamara a
API
            Navigator.push(
                context,
                MaterialPageRoute(builder: (context) => TelaInicial()),
            );
        },
    ),

```



```

style: ElevatedButton.styleFrom(
  backgroundColor: const Color.fromRGBO(188, 192, 198, 1),
  foregroundColor: Colors.black,
  minimumSize: const Size(double.infinity, 50),
  shape: RoundedRectangleBorder(
    borderRadius: BorderRadius.circular(10),
  ),
  shadowColor: Colors.black.withOpacity(0.1),
  elevation: 5,
),
child: const Text("Entrar"),
),
const SizedBox(height: 25),

// nao tem conta?
Column(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    const Text(
      "Não tem uma conta?",
      style: TextStyle(color: Colors.white),
    ),
    TextButton(
      onPressed: () {
        Navigator.push(
          context,
          MaterialPageRoute(builder: (context) => TelaCadastro()),
        );
      },
      child: const Text(
        "Cadastrar",
        style: TextStyle(color: Colors.white, fontWeight: FontWeight.bold),
      ),
    ),
  ],
),
const SizedBox(height: 15),
],
),
);
}
}

```

TelaInicial

Bem simples, pois já programamos sua estrutura, o mais difícil já foi.

```
class TelaInicial extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return BaseInicial(
      child: MostraProdutos(),
      //child: MostraProdutos(produtos: []), -> quando tiver a api, passar como
parametro
    );
  }
}
```

TelaProduto

Aqui também usamos o StatefulWidget. Essa foi bem complicada pois o visual é bem diferente dos demais e o alinhamento da imagem também.

```
class TelaProduto extends StatefulWidget {
  // parametros necessarios para poder apresentar o produto
  final String nome;
  final String imagem;
  final String descricao;
  final String linkReceita;

  //construtor que pede os parametros
  const TelaProduto({Key? key, required this.nome, required this.imagem, required
this.descricao, required this.linkReceita}) : super(key: key);

  @override
  State<TelaProduto> createState() => _TelaProduto();
}

class _TelaProduto extends State<TelaProduto>{

  void _adicionarOuRemoverFavoritos() { // a implementar
    final String usuario;
    widget.nome; //com widget pois o pega o parametro da classe q ele estende
  }

  void abrirPaginaWeb(String url) async {
    final Uri uri = Uri.parse(url);
    try {
```

```

        if (await canLaunchUrl(uri)) {
            await launchUrl(
                uri,
                mode: LaunchMode.externalApplication, // abre no navegador
            );
        }
        else {
            throw 'Não foi possível abrir $url';
        }
    }
}

catch (e) {
    print('Erro ao tentar abrir o URL: $e'); // imprime o erro no console
}
}

void _removerOuAdicionarWishlist() { // a implementar
    final String usuario;
    widget.nome; //com widget pois o pega o parametro da classe q ele estende
}

@override
Widget build(BuildContext context) {
    final largura = MediaQuery.of(context).size.width;

    return BaseInicial(
        child: SingleChildScrollView(
            child: Column(
                children: [
                    Stack(
                        clipBehavior: Clip.none, // permite que a imagem saia do Stack
                        children: [

                            Padding(
                                padding: EdgeInsetsGeometry.all(20),
                                child: Row(
                                    mainAxisAlignment: MainAxisAlignment.spaceBetween, // espaça os
elementos

                                    children: [
                                        // posiciona o nome do produto
                                        Positioned(
                                            top: 20,
                                            left: 20,
                                            child: Text(
                                                widget.nome,

```



```

    ),
  ],
),
const SizedBox(height: 20),

// box/container cinza que fica em baixo com descricao e botoes
Container(
  height: MediaQuery.of(context).size.height * 0.5, // metade da tela
  width: double.infinity,
  padding: const EdgeInsets.all(20),
  decoration: const BoxDecoration(
    color: Color.fromRGBO(255, 255, 255, 0.4),
    borderRadius: BorderRadius.only( // bordas somente em
cima
      topLeft: Radius.circular(20),
      topRight: Radius.circular(20),
    ),
  ),
  child: Column (
    children: [
      // descricao
      Text(
        widget.descricao,
        style: const TextStyle(
          color: Colors.black,
          fontSize: 16,
        ),
      ),
      const SizedBox(height: 25),

      // botao de receita
      ElevatedButton(
        onPressed: () {
          abrirPaginaWeb(widget.linkReceita);
        },
        style: ElevatedButton.styleFrom(
          backgroundColor: Colors.transparent, // fundo
transparente
          foregroundColor: Colors.black, // cor do texto e
ícone
          minimumSize: const Size(double.infinity, 50),
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(20),
            side: const BorderSide(color: Colors.black), // borda preta

```

```

        ),
        elevation: 0, // remove sombra
    ),
    child: const Text("Receita",),
  ),
  const SizedBox(height: 15),

  // linha que fica os ultimos dois botoes
  Row(
    children: [
      //botao wishlist
      Expanded( // preenche o espaço disponível
        //proporcionalmente
        child: ElevatedButton(
          onPressed: () {
            _removerOuAdicionarWishlist();
          },
          style: ElevatedButton.styleFrom(
            backgroundColor: Colors.transparent, // fundo
            //transparente
            foregroundColor: Colors.black, // cor do
            //texto e ícone
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(20),
              side: const BorderSide(color: Colors.black), // borda
            //preta
            ),
            elevation: 0, // remove
            //sombra
          ),
          child: const Text("Wishlist/Remover",),
        ),
      ),
      const SizedBox(width: 10),

      // botao degustar que leva p tela de avaliar
      Expanded(child: ElevatedButton(
        onPressed: () {
          Navigator.push(
            context,
            MaterialPageRoute(builder: (context) => TelaAvaliar())
          );
        },
        style: ElevatedButton.styleFrom(

```

```

        backgroundColor: Colors.transparent, // fundo
transparente
        foregroundColor: Colors.black, // cor do
texto e ícone
        shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(20),
            side: const BorderSide(color: Colors.black), // borda preta
        ),
        elevation: 0, // remove
sombra
    ),
    child: const Text("Degustar",),
  ),
),
],
)
]
),
),
],
),
),
);
}
}

```

TelaPesquisar

```

class TelaPesquisar extends StatefulWidget {
  @override
  State<TelaPesquisar> createState() => _TelaPesquisarState();
}

class _TelaPesquisarState extends State<TelaPesquisar> {
  final TextEditingController _pesquisaController = TextEditingController();

  @override
  Widget build(BuildContext context) {
    return BaseInicial(
      child: Column(
        children: [
          Padding(
            padding: const EdgeInsets.all(15),
            child: TextField(

```

```

        controller: _pesquisaController, // para passarmos como parametro
da lista de produtos
        decoration: InputDecoration(
            hintText: "Pesquisar", // texto que funciona como hint
(dica) para sugerir o que o usuario deve escrever no campo input
            prefixIcon: Icon( // icone de lupa do proprio
flutter
                Icons.search,
                color: const Color.fromARGB(255, 0, 0, 1)),
            filled: true, // preenche com a cor absixo
            fillColor: Colors.white.withOpacity(0.2),
            border: OutlineInputBorder(
                borderRadius: BorderRadius.circular(20),
                borderSide: BorderSide.none,
            ),
        ),
        style: TextStyle(
            color: const Color.fromARGB(255, 0, 0, 1)
        ),
    ),
),
Expanded( // faz o GridView de
mostraProdutos ocupar o resto da tela
    child: MostraProdutos(),
),
],
),
);
}
}

```

TelaConta

Não comentei muito essa, pois já vimos tudo daqui, aí não fica repetitivo e serve de prática.

```

class TelaConta extends StatelessWidget {
    @override
    Widget build(BuildContext context) {
        return BaseInicial(
            child: SingleChildScrollView(
                child: Column(
                    children: [
                        Padding(
                            padding: EdgeInsetsGeometry.all(15),
                            child: Row(

```



```

        mainAxisAlignment: MainAxisAlignment.start,
        children: [
          Text(
            "Conta",
            style: TextStyle(
              fontSize: 40,
              fontWeight: FontWeight.bold,
              color: Colors.white,
            ),
          ),
        ],
      ),
    ),
    const SizedBox(height: 25),

    Padding(
      padding: EdgeInsetsGeometry.all(20),
      child: Column(
        children: [
          //favoritos
          Container(
            padding: const EdgeInsets.all(10),
            decoration: BoxDecoration(
              color: const Color.fromARGB(255, 96, 106, 121), // cor de
fundo do container
              border: Border(
                top: BorderSide(color: Colors.white, width: 2), // temos
borda so em cima e em baixo
                bottom: BorderSide(color: Colors.white, width: 2),
              ),
            ),
            child: Row( // container fica como linha
              children: [
                Text(
                  "Favoritos",
                  style: TextStyle(
                    fontSize: 20,
                    fontWeight: FontWeight.bold,
                    color: Colors.white,
                  ),
                ),
              ],
            ),
          ),
        ],
      ),
    ),
  ),

```

```

MostraProdutos(),
const SizedBox(height: 25),

//wishlist
GestureDetector(
  onTap: () {
    Navigator.push(
      context,
      MaterialPageRoute(builder: (context) => TelaWishlist())
    );
  },
  child: Container(
    padding: const EdgeInsets.all(10),
    decoration: BoxDecoration(
      color: const Color.fromARGB(255, 96, 106, 121), // cor de fundo
      border: Border(
        top: BorderSide(color: Colors.white, width: 2),
        bottom: BorderSide(color: Colors.white, width: 2),
      ),
    ),
    child: Row(
      children: [
        Text(
          "Wishlist",
          style: TextStyle(
            fontSize: 20,
            fontWeight: FontWeight.bold,
            color: Colors.white,
          ),
        ),
        const Spacer(), // empurra o ícone para a
direita

        const Icon(
          Icons.arrow_forward_ios, // seta horizontal
          color: Colors.white,
          size: 20,
        ),
      ],
    ),
  ),
  MostraProdutos(),
  const SizedBox(height: 25),

```

```
//degustados
GestureDetector(
  onTap: () {
    Navigator.push(
      context,
      MaterialPageRoute(builder: (context) => TelaDegustados())
    );
  },
  child: Container(
    padding: const EdgeInsets.all(10),
    decoration: BoxDecoration(
      color: const Color.fromARGB(255, 96, 106, 121), // cor de fundo
      border: Border(
        top: BorderSide(color: Colors.white, width: 2),
        bottom: BorderSide(color: Colors.white, width: 2),
      ),
    ),
    child: Row(
      children: [
        Text(
          "Degustados",
          style: TextStyle(
            fontSize: 20,
            fontWeight: FontWeight.bold,
            color: Colors.white,
          ),
        ),
        const Spacer(), // empurra o ícone para a direita
        const Icon(
          Icons.arrow_forward_ios, // seta horizontal
          color: Colors.white,
          size: 20,
        ),
      ],
    ),
  ),
),
MostraProdutos(),
],
),
],
),
),
```

```
);
}
}
```

TelaConfiguracoes

Aqui também já vimos tudo, se quiser pular.

```
class TelaConfiguracoes extends StatefulWidget {
  @override
  State<TelaConfiguracoes> createState() => _TelaConfiguracoes();
}

class _TelaConfiguracoes extends State<TelaConfiguracoes> {
  // declarando controladores para pegar o texto de cada campo.
  final TextEditingController _userController = TextEditingController();
  final TextEditingController _emailController = TextEditingController();
  final TextEditingController _passwordController = TextEditingController();

  // limpa os controladores quando a tela é chamada
  @override
  void dispose() {
    _userController.dispose();
    _emailController.dispose();
    _passwordController.dispose();
    super.dispose();
  }

  _sairConta() {
    // a implementar
  }

  _excluirConta() {
    // a implementar
  }

  // função que será chamada quando o botão "Salvar" for pressionado e definira as
  // variáveis
  void _salvarUsuario() {
    final String usuario = _userController.text;
    final String email = _emailController.text;
    final String senha = _passwordController.text;
    // aqui ele chama outra funcao q manda as variaveis p banco de dados
  }
}
```

```

@override
Widget build(BuildContext context) {
  return BaseInicial(
    child: BaseBloqueio(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,          // ocupa o mínimo de espaço
vertical.

        children: [
          Row(
            mainAxisAlignment: MainAxisAlignment.start,
            children: [
              Text(
                "Configurações",
                style: TextStyle(
                  fontSize: 36,
                  fontWeight: FontWeight.bold,
                  color: Colors.white,
                ),
              ),
            ],
          ),
          const SizedBox(height: 20),

          TextField(
            controller: _userController,
            decoration: InputDecoration(
              labelText: "User",
              labelStyle: TextStyle(color: Colors.white),
              enabledBorder: UnderlineInputBorder(
                borderSide: BorderSide(color: Colors.white),
              ),
              focusedBorder: UnderlineInputBorder(
                borderSide: BorderSide(color: Colors.white),
              ),
            ),
            style: TextStyle(color: Colors.white),
          ),
          const SizedBox(height: 15),

          TextField(
            controller: _emailController,
            decoration: InputDecoration(
              labelText: "E-mail",

```

```

        labelStyle: TextStyle(color: Colors.white),
        enabledBorder: UnderlineInputBorder(
            borderSide: BorderSide(color: Colors.white),
        ),
        focusedBorder: UnderlineInputBorder(
            borderSide: BorderSide(color: Colors.white),
        ),
    ),
    style: TextStyle(color: Colors.white),
),
const SizedBox(height: 15),

TextField(
    controller: _passwordController,
    decoration: InputDecoration(
        labelText: "Password",
        labelStyle: TextStyle(color: Colors.white),
        enabledBorder: UnderlineInputBorder(
            borderSide: BorderSide(color: Colors.white),
        ),
        focusedBorder: UnderlineInputBorder(
            borderSide: BorderSide(color: Colors.white),
        ),
    ),
    style: TextStyle(color: Colors.white),
),
const SizedBox(height: 25),

//botao sair
ElevatedButton(
    onPressed: () {
        _sairConta();
        Navigator.push(
            context,
            MaterialPageRoute(builder: (context) => TelaBloqueio()),
        );
    },
    style: ElevatedButton.styleFrom(
        backgroundColor: Colors.transparent, //
        foregroundColor: const Color.fromARGB(255, 255, 255, 255), //
        minimumSize: const Size(double.infinity, 50),
        shape: RoundedRectangleBorder(

```

```

        borderRadius: BorderRadius.circular(20),
        side: const BorderSide(color: Color.fromARGB(255, 255, 255, 255)), //
borda preta
    ),
    elevation: 0,
// remove sombra
    ),
    child: const Text("Sair",),
    ),
    const SizedBox(height: 15),

    //botao excluir conta
    ElevatedButton(
      onPressed: () {
        _excluirConta();
        Navigator.push(
          context,
          MaterialPageRoute(builder: (context) => TelaBloqueio()),
        );
      },
      style: ElevatedButton.styleFrom(
        backgroundColor: Colors.transparent,
        foregroundColor: const Color.fromARGB(255, 255, 255, 255),
        minimumSize: const Size(double.infinity, 50),
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(20),
          side: const BorderSide(color: Color.fromARGB(255, 255, 255, 255)),
        ),
        elevation: 0,
      ),
      child: const Text("Excluir conta",),
    ),
    const SizedBox(height: 15),

    // botao de salvar
    ElevatedButton(
      onPressed: () {
        _salvarUsuario();
        Navigator.push(
          context,
          MaterialPageRoute(builder: (context) => TelaInicial()),
        );
      },
      style: ElevatedButton.styleFrom(

```

```

        backgroundColor: const Color.fromRGBO(188, 192, 198, 1),
        foregroundColor: Colors.black,
        minimumSize: const Size(double.infinity, 50),
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(20),
        ),
        shadowColor: Colors.black.withOpacity(0.1),
        elevation: 5,
      ),
      child: const Text("Salvar"),
    ),
    const SizedBox(height: 15),
  ],
),
),
);
}
}

```

TelaWishlist - TelaDegustados

A tela wishlist e degustados tem exatamente a mesma estrutura, são bem básicas, logo, só colocarei aqui a de wishlist.

```

class TelaWishlist extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return BaseInicial(
      child: Column(
        children: [
          Padding(
            padding: EdgeInsetsGeometry.all(15),
            child: Row(
              mainAxisAlignment: MainAxisAlignment.start,
              children: [
                Text(
                  "Wishlist",
                  style: TextStyle(
                    fontSize: 40,
                    fontWeight: FontWeight.bold,
                    color: Colors.white,
                  ),
                ),
              ],
            ),
          ],
        ),
      ),
    ),
  ),
)

```



```

        const SizedBox(height: 25),

        MostraProdutos(),
        //child: MostraProdutos(produtos: []),,]
    ]
)
);
}
}

```

TelaAvaliar

E, por último, finalmente, a tela de avaliar.

```

class TelaAvaliar extends StatefulWidget {
  @override
  State<TelaAvaliar> createState() => _TelaAvaliar();
}

class _TelaAvaliar extends State<TelaAvaliar> {
  // declarando controladores para pegar o texto de cada campo.
  final TextEditingController _userController = TextEditingController();
  final TextEditingController _notaController = TextEditingController();
  final TextEditingController _descricaoController = TextEditingController();
  int estrelasSelecionadas = 0;          // adiciona como variável da State

  // limpa os controladores quando a tela é chamada
  @override
  void dispose() {
    _userController.dispose();
    _notaController.dispose();
    _descricaoController.dispose();
    super.dispose();
  }

  // função que será chamada quando o botão "Cadastrar" for pressionado e definira as
  // variáveis
  void _salvarAvaliacao() {
    final String usuario = _userController.text;
    final String nota = _notaController.text;
    final String descricao = _descricaoController.text;
    // aqui ele chama outra funcao q manda as variaveis p banco de dados
  }

  @override

```

```

Widget build(BuildContext context) {
  return BaseInicial(
    child: BaseBloqueio(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,          // ocupa o mínimo de espaço
vertical.
        children: [
          Row(
            mainAxisAlignment: MainAxisAlignment.start,
            children: [
              Text(
                "Degustar",
                style: TextStyle(
                  fontSize: 36,
                  fontWeight: FontWeight.bold,
                  color: Colors.white,
                ),
              ),
            ],
          ),
          const SizedBox(height: 20),

          //nota
          Row(
            mainAxisAlignment: MainAxisAlignment.center,
            children: List.generate(5, (index) {           // vai gerar 5 estrelas (return
IconButton) com 5 indices (0-4)
              return IconButton(
                onPressed: () {                             // quando pressionar a
selecionada
                  setState(() {
                    estrelasSelecionadas = index + 1;      // define
quantas estrelas estão selecionadas
                    _notaController.text = estrelasSelecionadas.toString(); //
atualiza o controlador de nota referente a estrelas selecionadas
                  });
                },
                icon: Icon(
                  // se o indice da estrela for 4, mas as estrelas selecionadas forem
somente duas, a estrela nao ser apreenchida
                  // se o indice for menor que as estrelas selecionadas, use-se
Icons.star
                  // se o indice for MAIOR que as estrelas selecionadas, use-se
Icons.star_border

```

```

        index < estrelasSelecionadas ? Icons.star : Icons.star_border,
        color: const Color.fromARGB(255, 255, 255, 255),          // cor da
estrela cheia
        size: 36,
      ),
    );
  }},
),
const SizedBox(height: 15),

TextField(
  controller: _descricaoController,
  decoration: InputDecoration(
    labelText: "Descrição",
    labelStyle: TextStyle(color: Colors.white),
    enabledBorder: UnderlineInputBorder(
      borderSide: BorderSide(color: Colors.white),
    ),
    focusedBorder: UnderlineInputBorder(
      borderSide: BorderSide(color: Colors.white),
    ),
  ),
  style: TextStyle(color: Colors.white),
),
const SizedBox(height: 25),

// botao de salvar
ElevatedButton(
  onPressed: () {
    _salvarAvaliacao();
    Navigator.push(
      context,
      MaterialPageRoute(builder: (context) => TelaInicial()),
    );
  },
  style: ElevatedButton.styleFrom(
    backgroundColor: const Color.fromRGBO(188, 192, 198, 1),
    foregroundColor: Colors.black,
    minimumSize: const Size(double.infinity, 50),
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(20),
    ),
    shadowColor: Colors.black.withOpacity(0.1),
    elevation: 5,
  ),
),

```

```
        ),  
        child: const Text("Degustado"),  
      ),  
      const SizedBox(height: 15),  
    ],  
  ),  
),  
);  
}
```

Foi isso! Espero que tenha ajudado, qualquer dúvida me procure.